

PYTHON PROGRAMMING (1BPLC105B/205B)

Module-1:

The way of the program: The single most important skill for a computer scientist is problem solving. Problem solving means the ability to formulate problems, think creatively about solutions, and express a solution clearly and accurately. As it turns out, the process of learning to program is an excellent opportunity to practice problem-solving skills.

The Python programming language: it is a widely used general-purpose, high level programming language. It was initially designed by **Guido van Rossum** in **1991** and developed by Python Software Foundation. It was mainly developed for emphasis on code readability, and its syntax allows programmers to express concepts in fewer lines of code.

what is a program?

A program is a sequence of instructions that specifies how to perform a computation. The computation might be something mathematical, such as solving a system of equations or finding the roots of a polynomial, but it can also be a symbolic computation, such as searching and replacing text in a document or (strangely enough) compiling a program.

The details look different in different languages, but a few basic instructions appear in just about every language:

input Get data from the keyboard, a file, or some other device such as a sensor.

output Display data on the screen or send data to a file or other device such as a motor.

math Perform basic mathematical operations like addition and multiplication.

conditional execution Check for certain conditions and execute the appropriate sequence of statements.

repetition Perform some action repeatedly, usually with some variation.

What is debugging?

Programming errors are called **bugs** and the process of tracking them down and correcting them is called **debugging**.

Debugging is the process of finding and fixing errors or bugs in the source code of any software. When software does not work as expected, computer programmers study the code to determine why any errors occurred. They use debugging tools to run the software in a controlled environment, check the code step by step, and analyse and fix the issue.

Errors: An error describes any issue that arises unexpectedly that cause a computer to not function properly. Computers can encounter either software errors or hardware errors. Syntax errors, also known as parsing errors.

Syntax errors: Syntax refers to the structure of a program and the rules about that structure. Python can only execute a program if the program is syntactically correct; otherwise, the process fails and returns an error message, when the interpreter parses the Python code to convert it into bytecode and on encountering any invalid syntax in the parsing stage, it throws an error called the SyntaxError.

Syntax errors often arise from simple mistakes like missing colons, incorrect indentation, or mismatched parentheses.

Examples:

Types of Error	Incorrect Syntax	Error message	Correct Syntax
Missing Colon	<pre>if x == 10 print("x is 10")</pre>	SyntaxError: invalid syntax	<pre>if x == 10: print("x is 10")</pre>
Incorrect Indentation	<pre>def greet(): print("Hello, World!")</pre>	IndentationError: expected an indented block	<pre>def greet(): print("Hello, World!")</pre>
Incorrect String Delimiters	<pre>print('Hello, World!')</pre>	SyntaxError: EOL while scanning string literal	<pre>print("Hello, World!")</pre>
Mismatched Parentheses	<pre>print("Hello, World!"</pre>	SyntaxError: unexpected EOF while parsing	<pre>print("Hello, World!")</pre>
Using Reserved Keywords	<pre>class = "Introduction to Python"</pre>	SyntaxError: invalid syntax	<pre>course_name = "Introduction to Python"</pre>
Missing Parentheses in Function Calls	<pre>print "Hello, World!"</pre>	Missing parentheses in call to 'print'. Did you mean print("Hello, World!")?	<pre>print("Hello, World!")</pre>
Misusing Operators	<pre>result = 5 +</pre>	SyntaxError: invalid syntax	<pre>result = 5 + 3</pre>
Unexpected Indentation	<pre>def add(a, b): return a + b</pre>	IndentationError: unindent does not	<pre>def add(a, b): return a + b</pre>

	<code>print(add(2, 3))</code>	match any outer indentation level	<code>print(add(2, 3))</code>
Using Commas Incorrectly	<code>def greet(name,): print("Hello, " + name)</code>	SyntaxError: invalid syntax	<code>def greet(name): print("Hello, " + name)</code>
Missing or Extra Characters	<code>my_list = [1, 2, 3, print(my_list)</code>	SyntaxError: unexpected EOF while parsing	<code>my_list = [1, 2, 3] print(my_list)</code>

Runtime errors: Runtime error occurs when your syntax is correct but the compiler (or interpreter in case of Python), is still not able to run the code due to an error. the error does not appear until you run the program. These errors are also called exceptions because they usually indicate that something exceptional (and bad) has happened.

Runtime errors occur due to the computing environment in which the software code runs. Examples include insufficient memory space or stack overflow. You can resolve runtime errors by surrounding statements in try-catch blocks or logging the exception with an appropriate message.

Example :-

```
a = 5
```

```
b = 0
```

```
c = a / b
```

If you run the above code, you will get this error.

Traceback (most recent call last):

```
File "/mnt/sol.py", line 3, in <module>
```

```
    c = a / b
```

```
    ~~~~
```

```
ZeroDivisionError: division by zero
```

Semantic errors: the semantic error, also called a logic error. If there is a semantic error in your program, it will run successfully in the sense that the computer will not generate any error messages. However, your program will not do the right thing. It will do something else. Specifically, it will do what you told it to do, not what you wanted it to do.

1. Type Mismatches

Example 1: Concatenating a string and an integer

```
A = 10
```

```
B = "hello"
```

Result = A + B

```
print("Result is: ", Result)
```

2: Forgetting type conversion with user input

```
num1 = input('Enter a number: ')
```

```
num2 = input('Enter another number: ')
```

```
sum = num1 + num2
```

```
print("The sum is:", sum)
```

Experimental debugging: It refers to the practical, hands-on process of using various tools and techniques to investigate and fix bugs in your code through testing and observation. Debugging is also like an experimental science. Once you have an idea what is going wrong, you modify your program and try again. If your hypothesis was correct, then you can predict the result of the modification, and you take a step closer to a working program. If your hypothesis was wrong, you have to come up with a new one.

Expressions and Statements:

Variables: Value: A **value** is one of the fundamental things — like a word or a number — that a program manipulates. Some values are 5 (the result when we add 2 + 3), and "Hello, World!".

Variables are fundamental building blocks that allow us to store, manipulate, and reuse data in our code. They are essential for creating dynamic and flexible programs.

A variable is like a labeled box where you can store data. Imagine you have a box labeled "age" and you put the number 25 in it. In Python, you would do this by writing: age = 25

Rules for Python Variable Names

When naming variables in Python, follow these rules:

1. Use only alphabets, numbers, and underscores (A-Z, a-z, 0-9, and _).
Example: user_age, total_count2
2. Start with a letter or underscore, not a number. Valid: _count, name1 Invalid: 1name
3. No spaces allowed. Valid: first_name Invalid: first name
4. Case-sensitive: age, Age, and AGE are three different variables.
5. Avoid using Python keywords (like if, for, while) as variable names.

Keywords: Keywords in Python are predefined words, that are not to be used as any other kind of identifiers like for example, variable names or function names. These

keywords are a part of the syntax in the language with a special meaning associated with them. They play a major role in Python code by defining its structure and logic.

Python keywords are reserved words with specific functions and restrictions in the language. Currently, Python has thirty-five keywords and four soft keywords. These keywords are always available in Python.

- There are 35 keywords and four soft keywords in Python.
- You can get a list of all keywords using keyword.kwlist from the keyword module.
- Soft keywords in Python act as keywords only in specific contexts.
- print and exec are keywords that have been deprecated and turned into functions in Python 3.

and	as	assert	break	class	continue
def	del	elif	else	except	exec
finally	for	from	global	if	import
in	is	lambda	nonlocal	not	or
pass	raise	return	try	while	with
yield	True	False	None		

Statements: A **statement** is an instruction that the Python interpreter can execute. We have only seen the assignment statement so far. Some other kinds of statements that we'll see shortly are while statements, for statements, if statements, and import statements.

The statement `x = expression` saves the value of the expression `2 + 3` in the variable `x`.

Expression: On the right side of the statement, there is an expression, `2 + 3`, that evaluates to the integer 5.

A combination of operands and operators is called an expression. The expression in Python produces some value or result after being interpreted by the Python interpreter. An expression in Python is a combination of operators and operands.

An example of expression can be :

```
x=x+10
```

`x=x+10`. In this expression, the first 10

10 is added to the variable `x`. After the addition is performed, the result is assigned to the variable `x`.

Example:

```
x = 25      # a statement
```

```
x = x + 10  # an expression
```

Evaluating expressions: A Python program contains one or more statements. A statement contains zero or more expressions. Python executes a statement by evaluating its expressions to values one by one. Python evaluates an expression by evaluating the sub-expressions and substituting their values.

Literal Expressions: A literal expression evaluates to the value it represents. Following are some examples of literal expressions:

EX:

10 ==> 10

'Welcome to Python' ==> 'Welcome to Python'

7.89 ==> 7.89

True ==> True

Binary Expressions: A binary expression consists of a binary operator applied to two operand expressions.

Ex:

2*6 ==> 12

8-6 ==> 2

8==8 ==> True

1000 > 100 ==> True

'hello' + 'world' ==> 'helloworld'

Unary Expressions: An unary expression contains one operator and single operand.

Ex:-

(5/5) ==> -1

-(3*4) ==> -12

-(2**4) ==> -16

-10 ==> -10

Compound Expressions: In a binary or unary expression, if an operand itself is an expression, such expression is known as a compound expression.

Ex:

3 * 2 + 1 ==> 7

$2 + 6 * 2 \Rightarrow 14$

$(2 + 6) * 2 \Rightarrow 16$

Variable Access Expressions: A variable access expressions allows us to access the value of a variable.

Ex:

```
>>>x = 10
```

```
>>>(x + 2) * 4
```

```
48
```

```
>>>x
```

```
10
```

Operators and operands:

Operators are special tokens that represent computations like addition, multiplication and division. The values the operator works on are called **operands**.

The following are all legal Python expressions whose meaning is more or less clear:

```
20 + 32
```

```
5 ** 2
```

```
(5 + 9) * (15 - 7)
```

The tokens `+`, `-`, and `*`, and the use of parentheses for grouping, mean in Python what they mean in mathematics. The asterisk (`*`) is the token for multiplication, and `**` is the token for exponentiation.

Type converter functions:

In programming, type conversion is the process of converting data of one type to another. For example: converting int data to str.

There are two types of type conversion in Python.

- Implicit Conversion - automatic type conversion
- Explicit Conversion - manual type conversion

Implicit Type Conversion: In certain situations, Python automatically converts one data type to another. This is known as implicit type conversion.

Example 1: Converting integer to float

Let's see an example where Python promotes the conversion of the lower data type (integer) to the higher data type (float) to avoid data loss.

```
integer_number = 123
float_number = 1.23
new_number = integer_number + float_number
# display new value and resulting data type
print("Value:",new_number)
print("Data Type:",type(new_number))
```

Output: Value: 124.23

Data Type: <class 'float'>

In the above example, we have created two variables: integer_number and float_number of int and float type respectively. Then we added these two variables and stored the result in new_number.

Explicit Type Conversion: In Explicit Type Conversion, users convert the data type of an object to required data type. We use the built-in functions like int(), float(), str(), etc to perform explicit type conversion. This type of conversion is also called typecasting because the user casts (changes) the data type of the objects.

Example 2: Addition of string and integer Using Explicit Conversion

```
num_string = '12'
num_integer = 23
print("Data type of num_string before Type Casting:",type(num_string))
# explicit type conversion
num_string = int(num_string)
print("Data type of num_string after Type Casting:",type(num_string))
num_sum = num_integer + num_string
print("Sum:",num_sum)
print("Data type of num_sum:",type(num_sum))
```

Output

Data type of num_string before Type Casting: <class 'str'>

Data type of num_string after Type Casting: <class 'int'>

Sum: 35

Data type of num_sum: <class 'int'>

Order of operations: When more than one operator appears in an expression, the order of evaluation depends on the rules of precedence. Python follows the same precedence rules for its mathematical operators that mathematics does. The acronym PEMDAS is a useful way to remember the order of operations.

1. Parentheses have the highest precedence and can be used to force an expression to evaluate in the order you want. Since expressions in parentheses are evaluated first, $2 * (3-1)$ is 4, and $(1+1)**(5-2)$ is 8. You can also use parentheses to make an expression easier to read, as in $(\text{minute} * 100) / 60$, even though it doesn't change the result.
2. Exponentiation has the next highest precedence, so $2**1+1$ is 3 and not 4, and $3*1**3$ is 3 and not 27.
3. Multiplication and both Division operators have the same precedence, which is higher than Addition and Subtraction, which also have the same precedence. So $2*3-1$ yields 5 rather than 4, and $5-2*2$ is 1, not 6.
4. Operators with the same precedence are evaluated from left-to-right. In algebra we say they are left-associative. So in the expression $6-3+2$, the subtraction happens first, yielding 3. We then add 2 to get the result 5. If the operations had been evaluated from right to left, the result would have been $6-(3+2)$, which is 1. (The acronym PEDMAS could mislead you to thinking that division has higher precedence than multiplication, and addition is done ahead of subtraction - don't be misled. Subtraction and addition are at the same precedence, and the left-to-right rule applies.)

Operations on strings: a string is a sequence of characters enclosed in quotes. It can include letters, numbers, symbols or spaces. Since Python has no separate character type, even a single character is treated as a string with length one. Strings are widely used for text handling and manipulation.

Basic Operations (Operators)

Operator	Description	Example	Output
----------	-------------	---------	--------

+	Concatenation: Combines two or more strings.	<code>s = "Hello" + " " + "World"</code>	"Hello World"
*	Repetition: Repeats a string a specified number of times.	<code>s = "ha" * 3</code>	"hahaha"
[]	Indexing: Accesses a character at a specific zero-based index.	<code>s = "Python"; print(s[0])</code>	"P"
[:]	Slicing: Extracts a substring from a range of indices.	<code>s = "Python"; print(s[1:4])</code>	"yth"
in	Membership: Checks if a substring exists within a string.	<code>print("Py" in "Python")</code>	True
not in	Membership: Checks if a substring does not exist.	<code>print("Java" not in "Python")</code>	True

String Methods

String methods are built-in functions that perform specific operations.

Method	Description	Example	Output
<code>.upper()</code>	Converts all characters to uppercase.	<code>s = "Hello"; print(s.upper())</code>	"HELLO"
<code>.lower()</code>	Converts all characters to lowercase.	<code>s = "WORLD"; print(s.lower())</code>	"world"
<code>.strip()</code>	Removes leading and trailing whitespace.	<code>s = " hi "; print(s.strip())</code>	"hi"

<code>.replace(old, new)</code>	Replaces all occurrences of <code>old</code> with <code>new</code> .	<code>s = "dog cat"; print(s.replace("dog", "bird"))</code>	<code>"bird cat"</code>
<code>.split(sep)</code>	Splits the string into a list of substrings using a delimiter.	<code>s = "a,b,c"; print(s.split(","))</code>	<code>['a', 'b', 'c']</code>
<code>.join(iterable)</code>	Joins elements of an iterable (e.g., list) into a string using the string as a separator.	<code>s = "-".join(["2025", "05"])</code>	<code>"2025-05"</code>
<code>.find(sub)</code>	Returns the lowest index of the substring <code>sub</code> , or -1 if not found.	<code>s = "hello"; print(s.find("l"))</code>	<code>2</code>
<code>.startswith(prefix)</code>	Checks if the string starts with the specified prefix.	<code>s = "Python"; print(s.startswith("Py"))</code>	<code>True</code>

Input : There is a built-in function in Python for getting input from the user:

```
name = input("Please enter your name: ")
```

The user of the program can enter the name and click OK, and when this happens the text that has been entered is returned from the input function, and in this case assigned to the variable `name`. Even if you asked the user to enter their age, you would get back a string like `"17"`. It would be your job, as the programmer, to convert that string into a int or a float, using the int or float converter functions

Composition: the most useful features of programming languages is their ability to take small building blocks and compose them into larger chunks.

$$\text{Area} = \pi R^2$$

Firstly, we'll do the four steps one at a time:

```
1 response = input("What is your radius? ")
```

```
2 r = float(response)
3 area = 3.14159 * r**2
4 print("The area is ", area)
```

Now let's compose the first two lines into a single line of code, and compose the second two lines into another line of code.

```
1 r = float( input("What is your radius? ") )
2 print("The area is ", 3.14159 * r**2)
```

If we really wanted to be tricky, we could write it all in one statement:

```
1 print("The area is ", 3.14159*float(input("What is your radius?"))**2)
```

Such compact code may not be most understandable for humans, but it does illustrate how we can compose bigger chunks from our building blocks.

The modulus operator: The modulus operator works on integers (and integer expressions) and gives the remainder when the first number is divided by the second. In Python, the modulus operator is a percent sign (%). The syntax is the same as for other operators. It has the same precedence as the multiplication operator.

```
>>> q = 7 // 3 # This is integer division operator
>>> print(q)
2
>>> r = 7 % 3
>>> print(r)
1
```

So 7 divided by 3 is 2 with a remainder of 1.

The modulus operator turns out to be surprisingly useful. For example, you can check whether one number is divisible by another—if $x \% y$ is zero, then x is divisible by y .

Also, you can extract the right-most digit or digits from a number. For example, $x \% 10$ yields the right-most digit of x (in base 10). Similarly $x \% 100$ yields the last two digits.

It is also extremely useful for doing conversions, say from seconds, to hours, minutes and seconds. So let's write a program to ask the user to enter some seconds, and we'll convert them into hours, minutes, and remaining seconds.

```
total_secs = int(input("How many seconds, in total?"))
hours = total_secs // 3600
secs_still_remaining = total_secs % 3600
minutes = secs_still_remaining // 60
```

```
secs_finally_remaining = secs_still_remaining % 60
print("Hrs=", hours, " mins=", minutes,
      "secs=", secs_finally_remaining)
```

Iteration: Iteration in Python is the process of repeating a block of code, typically using a for loop to traverse through elements of an iterable (like a list, tuple, string, or dictionary) or a while loop to repeat as long as a condition is true.

1. Iterating with a for loop (Definite Iteration)

The for loop is the most common and "Pythonic" way to iterate over a sequence of items. Python handles the iteration process automatically.

Over a list:

```
fruits = ["apple", "banana", "cherry"]
for fruit in fruits:
    print(fruit)
```

Output:

```
apple
banana
cherry
```

Over a string:

```
for char in "Python":
    print(char)
```

Output:

```
P
y
t
h
o
n
```

Using range() for a specific number of times:

```
for i in range(3):
    print("Iteration number:", i)
```

Output: Iteration number: 0
 Iteration number: 1
 Iteration number: 2

Over a dictionary (keys and values):

```
student_scores = {"Alice": 95, "Bob": 88, "Charlie": 92}
for name, score in student_scores.items():
    print(f'{name} scored {score}')
```

Output:

```
Alice scored 95
Bob scored 88
Charlie scored 92
```

2. Iterating with a while loop (Indefinite Iteration)

A while loop continues to execute as long as a given condition remains true.

```
count = 0
while count < 3:
    print("Count is:", count)
    count += 1 # Increment the iteration variable
```

Output:

```
Count is: 0
Count is: 1
Count is: 2
```

3. Manual Iteration using iter() and next()

Behind the scenes, Python's for loops use the iterator protocol, which involves iter() to create an iterator and next() to get the next item. This is how you would manually implement it:

```
numbers = [10, 20, 30]
# Get an iterator object from the iterable
my_iterator = iter(numbers)
# Manually get the next item each time
print(next(my_iterator))
print(next(my_iterator))
print(next(my_iterator))
```

Calling next() again would raise a StopIteration exception

Output:

10

20

30

Assignment: Iteration in Python involves repeatedly executing a block of code, typically using for or while loops. Assignment plays a crucial role in iteration, primarily by initializing loop control variables and updating values within the loop body during each pass.

Iteration with Assignment Examples

1. The for loop with a running total (accumulator)

This is a classic example where assignment is used to update a variable (an accumulator) during each iteration to compute a final result.

Initialize an accumulator variable

```
total_sum = 0
```

```
numbers = [1, 2, 3, 4]
```

Iterate through the list

```
for number in numbers:
```

```
    # Assignment: update total_sum in each iteration
```

```
    total_sum = total_sum + number
```

```
    # Abbreviated assignment can also be used: total_sum += number
```

```
    print(f'Current number: {number}, Running total: {total_sum}')
```

```
print(f'The final sum is: {total_sum}')
```

Output:

Current number: 1, Running total: 1

Current number: 2, Running total: 3

Current number: 3, Running total: 6

Current number: 4, Running total: 10

The final sum is: 10

Updating variables: Variable Updating in Python is a common pattern is updating a variable where its new value depends on its old value. The assignment statement works by first evaluating the expression on the right and then assigning that result to the variable on the left.

- **Initialization:** Variables must be initialized with a value before they can be updated. Attempting to use a variable that hasn't been defined will result in a `NameError`.
- **Increment/Decrement:** Adding to a variable is called an increment, while subtracting is called a decrement. Python offers augmented assignment operators (e.g., `+=`, `-=`) for conciseness. the for loop, the while statement, The Collatz $3n + 1$ sequence, tables, two-dimensional tables, break statement, continue statement, paired data, Nested Loops for Nested Data. Functions: Functions with arguments and return values.

The Collatz $3n + 1$ sequence : The collatz sequence is a conjecture in mathematics that follows a sequence. This sequence is defined below:

- The sequence begins with any positive integer, say n
- If the integer n is odd, the next number in sequence would be $3n+1$
- If the integer n is even, the next number in sequence would be $n/2$
- The sequence will continue until digit 1 is encountered

Ex:

Since 11 is odd, the second number in the sequence is

3
*
11
+
1
=
34
 $3*11+1=34$
, i.e., $3n+1$

Now we have [11, 34,...]

Since 34 is even, the next number is

34

/

2

=

17

34/2=17

, i.e., $n/2$

Now we have [11, 34, 17...]

17 is odd,

Algorithm

1. Read an integer, n
2. If n is even, $n=n/2$
3. If n is odd, $n=3n+1$
4. Repeat steps 2 and 3 until n becomes 1
5. `#include<stdio.h>`
6. `int main(){`
7. `int n = 11;`
8. `while(n > 1){`
9. `if(n % 2 == 0){ //for even numbers`
10. `n = n / 2;`
11. `printf("%d\n", n);`
12. `}`
13. `else{ //for odd numbers`
14. `n = n * 3 + 1;`
15. `printf("%d\n", n);`
16. `}`
17. `}`
18. `return 0;`
19. `}`

Output:

40

20

10

5

16

8

4

2

1

tables : Tables are a fundamental object type for representing data sets. A table can be viewed in two ways:

a sequence of named columns that each describe a single aspect of all entries in a data set, or a sequence of rows that each contain all information about a single entry in a data set.

In order to use tables, import all of the module called datascience, a module created for this text.

```
from datascience import *
```

Empty tables can be created using the Table function. An empty table is useful because it can be extended to contain new rows and columns.

Table()

The with_columns method on a table constructs a new table with additional labeled columns. Each column of a table is an array. To add one new column to a table, call with_columns with a label and an array. (The with_column method can be used with the same effect.)

Below, we begin each example with an empty table that has no columns.

```
Table().with_columns('Number of petals', make_array(8, 34, 5))
```

Number of petals

8

34

5

A two-dimensional table is a table where you read the value at the intersection of a row and a column. A multiplication table is a good example. Let's say you want to print a multiplication table for the values from 1 to 6. A good way to start is to write a loop that prints the multiples of 2, all on one line:

for i in range(1, 7):

print(2 * i, end=" ")

print()

Here we've used the range function, but made it start its sequence at 1. As the loop executes, the value of *i* changes from 1 to 6. When all the elements of the range have been assigned to *i*, the loop terminates. Each time through the loop, it displays the value of $2 * i$, followed by three spaces.

Again, the extra `end=" "` argument in the print function suppresses the newline, and uses three spaces instead.

After the loop completes, the call to print at line 3 finishes the current line, and starts a new line.

The output of the program is:

```
2 4 6 8 10 12
```

So far, so good. The next step is to encapsulate and generalize.

The break statement: The break statement is used to immediately leave the body of its loop. The next statement to be executed is the first one after the body:

```
for i in [12, 16, 17, 24, 29]:
```

```
    if i % 2 == 1: # If the number is odd
```

```
        break # ... immediately exit the loop
```

```
    print(i)
```

```
print("done")
```

This prints:

```
12
```

```
16
```

```
Done
```

Continue Statement:

Continue in Python is a loop control statement used to skip the remaining iterations of the loop and execute the next iteration. So, when we execute the Python continue statement in a loop, it skips the code within the loop for the current iteration and begins the next iteration of the loop.

We use the continue statement with an if statement to skip specific conditions that don't meet certain criteria. It is used when we want to ignore certain values in the loop and continue with the remaining iterations.

Syntax:

for variable in sequence:

 if condition:

 continue

 # remaining code

while condition:

 if condition_to_skip:

 continue

 # remaining code

Example:

```
for i in range(6):
```

```
    if i == 3:
```

```
        continue
```

```
    print("Number:", i)
```

Output:

Number: 0

Number: 1

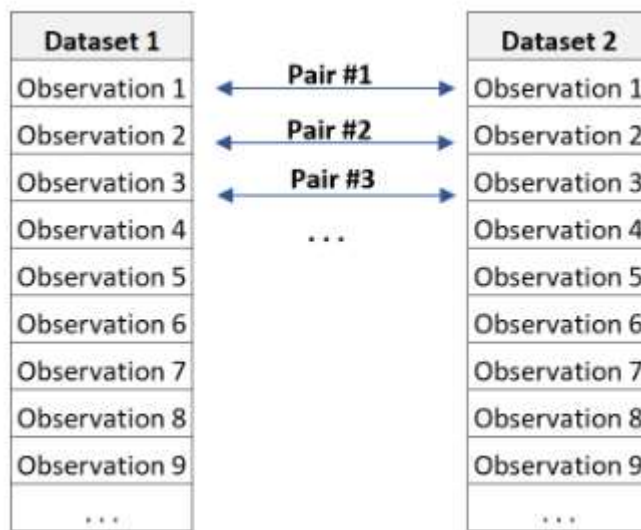
Number: 2

Number: 4

Number: 5

In this example, we skip printing the number 3 using the continue statement, and the loop moves on to print the remaining numbers from the range.

Paired data: When two datasets are of equal length and each observation in one dataset can be “paired” with an observation in another dataset, we call this paired data.



Paired data example:

```
year_born = ("Paris Hilton", 1981)
```

We can put many pairs into a list of pairs:

```
1 celebs = [("Brad Pitt", 1963), ("Jack Nicholson", 1937),  
2 ("Justin Bieber", 1994)]
```

Here is a quick sample of things we can do with structured data like this. First, print all the celebs:

```
1 print(celebs)
```

```
2 print(len(celebs))
```

```
[("Brad Pitt", 1963), ("Jack Nicholson", 1937), ("Justin Bieber", 1994)]
```

```
3
```

Nested Loops for Nested Data :

Nested loops are a fundamental Python concept used to efficiently traverse and process nested data structures like lists of lists, lists of dictionaries, or multi-dimensional arrays. The inner loop runs completely for every single iteration of the outer loop, allowing access to each individual data element.

Basic Structure and Logic

In a nested loop, you have an outer loop and one or more inner loops. The indentation is crucial in Python to define which code block belongs to which loop.

```
# A common use case: iterating over a list of lists (a 2D matrix)
```

```
nested_list = [  
    [1, 2, 3],  
    [4, 5, 6],  
    [7, 8, 9]  
]  
  
for row in nested_list:      # Outer loop iterates through each sublist (row)  
    for element in row:      # Inner loop iterates through each item in the current sublist  
        print(element, end=" ") # Process the element  
    print()                  # Print a newline character after each row is complete
```

Output:

```
1 2 3  
4 5 6  
7 8 9
```

The inner loop executes 3 times for each of the 3 iterations of the outer loop, for a total of 9 executions of the `print(element, end=" ")` statement.

Functions: Python Functions are a block of statements that does a specific task. A group of connected statements called a Python function is used to carry out a logical, computational, or evaluative operation.

Functions allow us to divide our program into more manageable and modular sections. As our program grows in size, functions allow us to keep it structured and manageable.

Advantages of Function Use are Boosting the Readability of Code and boosting the Reusability of Code. It is beneficial for the program to be brief, unique, and well-structured.

Functions with arguments: Python function arguments allow us to pass data to functions so they can work with different values each time. This makes our code reusable and flexible.

```
Ex: def greet(name, message):  
    print(f'{message}, {name}!')  
    greet("WsCube Tech", "Welcome")
```

Output:

```
Welcome, WsCube Tech!
```

Types of Function Arguments in Python

In Python, there are four built-in function arguments we use to call functions and perform certain tasks.

- Default Arguments
- Keyword Arguments
- Positional Arguments
- Arbitrary Arguments

return values: The Python return statement is a special statement that you can use inside a function or method to send the function's result back to the caller. A return statement consists of the return keyword followed by an optional return value.

The return statement serves two primary purposes:

It immediately exits the current function execution.

It sends a specified value (or None if no value is specified) back to the caller.

The basic syntax is straightforward:

```
def function_name(parameters):
```

```
    # Code to perform calculations or actions
```

```
    result = some_value_or_calculation
```

```
    return result
```

Important Questions:

- What is a program? Explain the process of debugging in Python, including syntax errors, runtime errors, and semantic errors.
- Describe the different types of errors in Python programming. Explain experimental debugging with an example.
- What are variables? Explain the rules for naming variables in Python. Describe the different data types in Python with examples.
- Evaluate the expression: `x = 5; y = 3; print(x + y * 2)` Explain the order of operations in Python.
- Write a Python program to take a string input from the user and print the length of the string. Explain the use of the modulus operator.

Iteration

- Explain the Collatz $3n + 1$ sequence with an example. Write a Python program to generate this sequence for a given number.
- Write a Python program to print a 2D table using nested loops. Explain the use of break and continue statements.
- Define a function in Python. Explain the difference between arguments and return values with an example.
- Write a Python function to calculate the area and perimeter of a rectangle. Explain the use of function arguments and return values.
- Explain the use of for loop and while statement in Python with examples.
- Describe the use of paired data and nested loops for nested data.
- Write a Python program to demonstrate the use of type converter functions.
- Explain the concept of composition in Python with an example.

Module-2:

Strings: Python string is a data type used to represent a sequence of characters. These characters can include letters, numbers, symbols, and whitespace. In Python, strings are immutable, meaning that once a string is created, its content cannot be changed. This immutability feature ensures that strings are consistent and secure throughout the program execution.

Strings in Python are enclosed within either single quotes (' '), double quotes (" "), or triple quotes ("\" \" or \"\" \"\"). Each type of quote can be used interchangeably, allowing flexibility in how strings are defined, particularly when a string itself contains quotes. For example:

Example:

Single quotes

```
single_quote_string = 'Hello, World!'
```

Double quotes

```
double_quote_string = "Hello, World!"
```

Triple quotes for multi-line strings or strings containing both single and double quotes

```
triple_quote_string = """Hello,World!"""
```

Working with strings as single things:

In Python, a **string** is a sequence of characters treated as a **single immutable object**. Working with strings as single things means performing operations on the entire string at once rather than processing character by character.

Python provides powerful **operators, built-in functions, and string methods** that support this approach

Characteristics of Python Strings

- Strings are **immutable**
- Stored as a single object in memory
- Support rich built-in operations
- Unicode support by default

Ex: s = "Data Structures"

Creating Strings

```
s1 = 'VTU'  
s2 = "Python"  
s3 = ""Data Structures""
```

Operations on Strings as Single Things

Length of a String

```
s = "Python"  
print(len(s))
```

String Concatenation

```
s1 = "Data"  
s2 = "Structures"  
print(s1 + " " + s2)
```

String Repetition

```
s = "VTU"  
print(s * 3)
```

String Comparison

```
print("apple" == "apple")  
print("cat" < "dog")
```

Working with the parts of a string:

strings are sequences of characters. Apart from treating strings as single entities, Python allows programmers to **access and manipulate specific parts of a string** efficiently using **indexing, slicing, and string methods**.

Working with parts of a string is useful for text processing, data validation, and pattern extraction.

String Indexing:

Each character in a string is assigned a unique index starting from **0**.

Example:

```
s = "PYTHON"
```

Character	P	Y	T	H	O	N
Index	0	1	2	3	4	5

Negative indexing is also supported:

| Index | -6 | -5 | -4 | -3 | -2 | -1 |

Example:

```
print(s[0]) # P
print(s[-1]) # N
```

String Slicing:

Slicing is used to extract a substring from a string.

Syntax:

```
string[start : end : step]
```

- start → starting index (inclusive)
- end → ending index (exclusive)
- step → increment value
- s = "PROGRAMMING"
- print(s[0:7]) # PROGRAM
- print(s[3:8]) # GRAMM
- print(s[:5]) # PROGR
- print(s[5:]) # AMMING
- print(s[::2]) # PORMIG
- print(s[::-1]) # Reverse string

Extracting Substrings Using Methods:

split(): Splits a string into parts based on a delimiter.

```
sentence = "Python is easy to learn"
words = sentence.split()
print(words)
```

partition(): Divides the string into three parts based on a separator.

```
s = "data-structures"
```

```
print(s.partition("-"))
```

find() and index(): Finds the position of a substring.

```
s = "datastructures"
```

```
print(s.find("struct"))
```

```
print(s.index("data"))
```

Strings: Length& Traversal

Length : The **length of a string** refers to the total number of characters present in the string.

- ☐ Helps in looping through strings
- ☐ Used in validation (password length, input size)
- ☐ Useful in indexing and slicing
- ☐ Important in algorithm design

In Python, the built-in function `len()` is used to find the length of a string.

Syntax: `len(string)`

Examples:

```
s = "Python"
```

```
print(len(s)) # Output: 6
```

```
s = "Data Structures"
```

```
print(len(s)) # Output: 15
```

Traversal refers to accessing each character of the string one by one for processing.

Python provides multiple ways to traverse a string efficiently.

Traversal Using for Loop

```
s = "VTU"
```

```
for ch in s:  
    print(ch)
```

4.2 Traversal Using Indexing

```
s = "Python"  
for i in range(len(s)):  
    print(s[i])  
Reverse Traversal of a String  
s = "PROGRAM"  
for ch in s[::-1]:  
    print(ch)
```

the for loop: the **for loop** is used to iterate over a **sequence of elements** such as strings, lists, tuples, ranges, and other iterable objects.

for loop works as a **for-each loop**, making it simple and readable.

- ☐ The loop picks the first element from the sequence
- ☐ Executes the loop body
- ☐ Repeats until all elements are processed
- ☐ Stops automatically when the sequence ends

Syntax of the for Loop:

```
for variable in sequence:  
    statement(s)
```

here,

- ☐ variable → takes one value at a time from the sequence
- ☐ sequence → string, list, tuple, range, etc.
- ☐ Indentation defines the loop body

for Loop with Strings:

Strings are sequences of characters. The for loop is commonly used to **traverse strings**.

Example:

```
s = "VTU"  
for ch in s:  
    print(ch)
```

Output:

```
V  
T  
U
```

for Loop with range()

The range() function generates a sequence of numbers.

Syntax:

```
range(start, stop, step)
```

Example:

```
for i in range(1, 6):  
    print(i)
```

Nested for Loop

A for loop inside another for loop is called a **nested for loop**.

Example:

```
for i in range(1, 4):  
    for j in range(1, 4):  
        print(i, j)
```

Slicing is a technique used in Python to extract a **part (substring)** of a string. Since strings are sequences, slicing allows access to multiple characters at once instead of one character at a time.

Slicing is widely used in text processing, data extraction, and string manipulation.

Syntax

string[start : end : step]

- start → Starting index (inclusive)
- end → Ending index (exclusive)
- step → Increment value (optional)

If any value is omitted, Python uses default values.

Example:

```
s = "PROGRAMMING"
print(s[0:7]) # PROGRAM
print(s[3:8]) # GRAMM
print(s[:5]) # PROGR
print(s[5:]) # AMMING
```

Negative Indexes

Negative indexes count from the end of the string.

```
s = "PYTHON"
print(s[-6:-1]) # PYTHO
print(s[-3:]) # HON
```

Questions

1. Explain slicing in Python with syntax
2. Write a Python program to reverse a string using slicing
3. Differentiate indexing and slicing
4. Write short notes on slicing with step value

String comparison is the process of comparing two or more strings to determine their relationship, such as equality or lexicographical (dictionary) order.

In Python, strings can be compared **directly as whole objects**, which supports the concept of working with strings as single things.

Python supports the following types of string comparison:

1. Equality comparison
2. Relational (lexicographical) comparison
3. Case-sensitive comparison
4. Case-insensitive comparison

Using == Operator

Checks whether two strings have the same content.

```
s1 = "Python"
s2 = "Python"
print(s1 == s2) # True
```

Using != Operator

Checks whether two strings are not equal.

```
print(s1 != s2) # False
```

Strings are Immutable

In Python, **strings are immutable**, meaning once a string is created, it cannot be modified.

Example:

```
s = "Python"
# s[0] = 'J' # Error
s = "J" + s[1:]
print(s) # Jython
```

Immutability improves security and performance.

The in and not in Operators

Used to check whether a substring exists within a string.

```
s = "datastructures"
print("data" in s) # True
```



```
print("tree" not in s) # True
```

The find() Function

Returns the index of the first occurrence of a substring. Returns -1 if not found.

```
s = "datastructures"  
print(s.find("struct"))  
print(s.find("tree"))
```

Looping and Counting in Strings

Example: Count number of vowels in a string.

```
s = "education"  
vowels = "aeiou"  
count = 0  
for ch in s:  
    if ch in vowels:  
        count += 1  
print(count)
```

Optional Parameters in String Methods

Some string methods accept optional parameters.

Example: find() with start and end positions.

```
s = "programming"  
print(s.find("m", 3))
```

Built-in find() Method

Syntax:

```
string.find(substring, start, end)
```

Example:

```
s = "python programming"  
print(s.find("program", 0, 10))
```

The split() Method

Splits a string into a list of substrings.

```
s = "Python is easy to learn"  
words = s.split()  
print(words)
```

Cleaning Up Your Strings

Cleaning strings includes removing spaces and unwanted characters.

Common methods:

- strip()
- lstrip()
- rstrip()
- replace()

Example:

```
s = " data structures "  
print(s.strip())
```

The format() Method

Used for formatting strings.

```
name = "VTU"  
year = 2025  
print("University: {}, Year: {}".format(name, year))
```

TUPLES:

Tuples for Grouping Data

A **tuple** is an ordered, immutable collection of elements.

```
t = (10, 20, 30)
```

Used to group related data.

Tuple Assignment

Multiple variables can be assigned at once.

```
a, b, c = (1, 2, 3)
```

Tuples as Return Values

Functions can return multiple values using tuples.

```
def calc(a, b):  
    return a+b, a-b
```

```
x, y = calc(10, 5)  
print(x, y)
```

Composability of Data Structures

Tuples can contain other data structures like lists and dictionaries.

```
data = ("VTU", [1, 2, 3], {"year": 2025})  
print(data)
```

Comparison: Strings vs Tuples

Feature	String	Tuple
Type	Characters	Any data type
Immutable	Yes	Yes
Use	Text data	Grouping data

LISTS:

1. List Values: A list is an ordered collection of elements enclosed in square brackets []. Lists can store elements of different data types.

- Lists maintain insertion order
- Lists allow duplicate values
- Lists are mutable (can be changed)

Example

```
my_list = [10, 20, 30, 'Python']  
print(my_list)
```

Output

```
[10, 20, 30, 'Python']
```

2. Accessing Elements

Accessing elements means retrieving values from a list using their **index position**. Index starts from 0

- Negative indexing starts from the end

Example

```
numbers = [10, 20, 30, 40]  
print(numbers[0])  
print(numbers[-1])
```

Output

10

40

3. List Length

The len() function returns the total number of elements in a list.

Example

```
numbers = [1, 2, 3, 4]
```

```
print(len(numbers))
```

Output

4

4. List Membership

Membership operators (in, not in) are used to check whether an element exists in a list.

Example

```
numbers = [10, 20, 30]
```

```
print(20 in numbers)
```

```
print(50 not in numbers)
```

Output

True

True

5. List Operations

List operations include **concatenation (+)** and **repetition (*)**.

Example

```
print([1, 2] + [3, 4])
```

```
print([1, 2] * 2)
```

Output

[1, 2, 3, 4]

[1, 2, 1, 2]

6. List Slices

Slicing extracts a portion of a list using the syntax `list[start:end]`.

Example

```
numbers = [10, 20, 30, 40, 50]
print(numbers[1:4])
```

Output

```
[20, 30, 40]
```

7. Lists are Mutable

Mutable means the contents of a list can be modified after creation.

Example

```
numbers = [10, 20, 30]
numbers[1] = 25
print(numbers)
```

Output

```
[10, 25, 30]
```

8. List Deletion

Elements can be removed from a list using `del`, `remove()`, or `pop()`.

Example

```
numbers = [10, 20, 30]
del numbers[0]
print(numbers)
```

Output

```
[20, 30]
```

9. Objects and References

Lists are objects in memory and variables store references to these objects.

Example

```
a = [1, 2, 3]
b = a
print(b)
```

Output

```
[1, 2, 3]
```

10. Aliasing

Aliasing occurs when two variables reference the same list object.

Example

```
a = [1, 2]
```

```
b = a
```

```
b.append(3)
```

```
print(a)
```

Output

```
[1, 2, 3]
```

11. Cloning Lists

Cloning creates a new independent copy of a list.

Example

```
a = [1, 2, 3]
```

```
b = a[:]
```

```
b.append(4)
```

```
print(a)
```

```
print(b)
```

Output

```
[1, 2, 3]
```

```
[1, 2, 3, 4]
```

12. Lists and For Loops

For loops are used to iterate through list elements.

Example

```
numbers = [10, 20, 30]
```

```
for n in numbers:
```

```
    print(n)
```

Output

```
10
```

```
20
```

```
30
```

13. List Parameters

Lists passed to functions can be modified inside the function.

Example

```
def add_item(lst):  
    lst.append(100)
```

```
numbers = [1, 2]  
add_item(numbers)  
print(numbers)
```

Output

```
[1, 2, 100]
```

14. List Methods

List methods are built-in functions that modify lists.

Example

```
numbers = [3, 1, 2]  
numbers.sort()  
print(numbers)
```

Output

```
[1, 2, 3]
```

15. Pure Functions and Modifiers

- **Pure functions** do not change the original list
- **Modifiers** change the original list

Example

```
numbers = [3, 1, 2]  
print(sorted(numbers))  
print(numbers)
```

Output

```
[1, 2, 3]
```

```
[3, 1, 2]
```

16. Functions that Produce Lists

Example

```
squares = [x*x for x in range(5)]  
print(squares)
```

Output

```
[0, 1, 4, 9, 16]
```

17. Strings and Lists

Example

```
word = 'Python'  
letters = list(word)  
print(letters)
```

Output

```
['P', 'y', 't', 'h', 'o', 'n']
```

18. List and Range

Example

```
numbers = list(range(1, 6))  
print(numbers)
```

Output

```
[1, 2, 3, 4, 5]
```

19. Nested Lists

A list inside another list is called a nested list.

Example

```
nested = [[1, 2], [3, 4]]  
print(nested[1][0])
```

Output

```
3
```

20. Matrices

Matrices are represented using nested lists.

Example

```
matrix = [[1,2,3],[4,5,6],[7,8,9]]
```

```
print(matrix[2][1])
```

Output

8

Important Questions:

- What is the difference between a string and a list in Python?
- Are strings mutable or immutable? Explain the implication of this characteristic.
- How do you find the length of a string?
- Explain string slicing. Provide an example of how to extract a substring from index 2 up to (but not including) index 5.
- How do the `in` and `not in` operators work with strings?
- Describe the purpose and usage of the built-in `find()` and `split()` methods.
- How can you loop through the characters of a string using a `for` loop?
- Provide examples of how to use the `format()` method for string formatting.
- What is the primary purpose of a tuple, and how does it differ from a list?
- Are tuples mutable or immutable?
- How do you access elements in a tuple using indexing and negative indexing?
- Explain "tuple assignment" and provide an example of how to swap two variable values using this technique.
- How can a function return multiple values? What data structure does Python use internally for this?
- Describe the concept of "composability" with respect to data structures (e.g., using a list as part of a tuple or vice versa).
- Are lists mutable or immutable?
- What is the difference between `append()` and `extend()` list methods?
- Explain the concepts of "aliasing" and "cloning" a list. Why is `new_list = old_list` not a true copy (cloning)?
- How are arguments passed to functions in Python (by value or by reference)? Relate this to how mutable objects like lists behave when passed as arguments.

- How do list slices work? How do you use slicing to reverse a list?
- Explain how to delete elements from a list using the `del` keyword, the `pop()` method, and the `remove()` method.
- Describe how to loop through a list and access both the elements and their indices.

Module-3

1. Definition of Dictionary

A Python dictionary serves as a collection of key-value pairs, in which each key is distinct and can encompass various object types, such as strings, numbers, or tuples. Correspondingly, each key can be linked to any type of object.

Dictionaries are frequently utilized for efficient data storage and retrieval. For example, they can be employed to organize contact details for friends and family. In such a dictionary, the keys might represent the names of individuals, while the values could denote their respective phone numbers, email addresses, and mailing addresses.

Dictionaries emerge as a potent data structure adaptable to an array of tasks. Several compelling reasons advocate for their use:

1. **Efficiency:** Dictionaries excel at rapid data retrieval and searching. They achieve this by employing a hash table for data storage, permitting constant-time access to values, regardless of dictionary size.
2. **Flexibility:** Dictionaries accommodate a wide range of data types, rendering them a versatile choice for data organization.
3. **Readability:** Dictionaries feature a user-friendly and easily comprehensible structure, making them well-suited for storing and retrieving data in a human-readable format.

Basic Syntax

To instantiate dictionaries, employ curly braces `{}`. Each key-value pair is demarcated by a colon `:`. For instance, the following code establishes a straightforward dictionary:

```
my_dictionary = {  
    "name": "Alice",  
    "age": 25,  
    "occupation": "Software Engineer"  
}
```

To access a value within a dictionary, employ the associated key. For instance, to retrieve Alice's age, one can use the following code:

```
age = my_dictionary["age"]
```

Creating & Accessing Dictionaries in Python

Two techniques exist for crafting dictionaries in Python:

1. Curly Braces: As previously demonstrated, dictionaries are constructed using curly braces, with key-value pairs separated by colons.
2. dict() Function: Alternatively, dictionaries can be generated through the dict() function. This function takes an iterable of key-value pairs as its argument and yields a dictionary with those pairs. For instance, the following code builds the same dictionary as the previous example, but employs the dict() function:

```
my_dictionary = dict([("name", "Alice"), ("age", 25), ("occupation", "Software Engineer")])
```

To access a dictionary's value, use the corresponding key enclosed in square brackets. For instance, to retrieve Alice's name, use the following code:

```
name = my_dictionary["name"]
```

2. Dictionary Operations

Dictionary operations are the basic actions that can be performed on dictionaries such as **creation, insertion, access, updating, deletion, membership testing, and traversal**.

Example

```
student = {"name": "Alice", "age": 20}  
student["age"] = 21  
print(student["age"])
```

Output

21

The del statement removes a key:value pair from a dictionary. For example, the following dictionary contains the names of various fruits and the number of each fruit in stock:

```
>>> inventory = {"apples": 430, "bananas": 312, "oranges": 525, "pears": 217}  
>>> print(inventory)  
{'pears': 217, 'apples': 430, 'oranges': 525, 'bananas': 312}
```

If someone buys all of the bananas, we can remove the entry from the dictionary:

```
>>> del inventory["bananas"]
```

```
>>> print(inventory)
```

```
{'apples': 430, 'oranges': 525, 'pears': 217}
```

If we then try to see how many bananas we have, we get an error (because, yes, we have no bananas). (Try this!)

Or if we're expecting more bananas soon, we might just change the value associated with bananas:

```
>>> inventory["bananas"] = 0
```

```
>>> print(inventory)
```

```
{'pears': 217, 'apples': 430, 'oranges': 525, 'bananas': 0}
```

A new shipment of bananas arriving could be handled like this:

```
>>> inventory["bananas"] += 200
```

```
>>> print(inventory)
```

```
{'pears': 0, 'apples': 430, 'oranges': 525, 'bananas': 512}
```

The len function also works on dictionaries; it returns the number of key:value pairs:

```
>>> len(inventory)
```

```
4
```

3. Dictionary Methods

Dictionary methods are **built-in functions** provided by Python to perform common operations on dictionaries, such as accessing keys and values, safely retrieving data, updating content, removing elements, and copying dictionaries.

Common Methods

- get()
- keys()
- values()
- items()
- update()
- pop()
- clear()

Example

```
student = {"name": "Alice"}  
print(student.get("name"))
```

Output

Alice

get()

The `get()` method retrieves the value linked to a particular key. This method takes two arguments: the key for which the value is sought and an optional default value. If the key is absent in the dictionary, the `get()` method returns the default value. For instance:

```
name = my_dictionary.get("name", "Unknown")
```

keys()

The `keys()` method produces a sorted list of all keys present in the dictionary. This list is arranged in a sorted order. For instance:

```
for key in my_dictionary.keys():  
    print(key)
```

values()

The `values()` method furnishes a list of all values in the dictionary, preserving the order identical to the keys. For example:

```
for value in my_dictionary.values():  
    print(value)
```

items()

The `items()` method assembles a list of tuples, where each tuple embodies a key-value pair from the dictionary. The list preserves the same order as the keys. Here's an illustration:

```
for key, value in my_dictionary.items():  
    print(f'{key}: {value}')
```

update()

The `update()` method integrates the contents of one dictionary into another. It accepts one argument: a dictionary containing key-value pairs to append to the existing dictionary. For instance:

```
new_dictionary = {"favorite_color": "blue"}  
my_dictionary.update(new_dictionary)
```

pop()

The pop() method eliminates an item from the dictionary and returns the associated value. It takes two arguments: the key of the item to remove and an optional default value. Should the key not exist, the pop() method returns the default value or raises a KeyError if no default is provided. For example:

```
name = my_dictionary.pop("name")
```

popitem()

The popitem() method withdraws a random item from the dictionary and delivers it as a tuple comprising the key and value of the removed item. For instance:

```
key, value = my_dictionary.popitem()
```

clear()

The clear() method purges all items from the dictionary. For example:

```
my_dictionary.clear()
```

4. Aliasing: **Aliasing** occurs when **two or more variables refer to the same dictionary object in memory**, so changes made through one variable are reflected in the other.

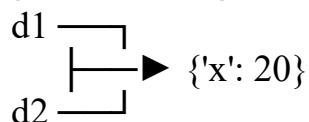
Example

```
d1 = {"x": 10}
d2 = d1
d2["x"] = 20
print(d1)
```

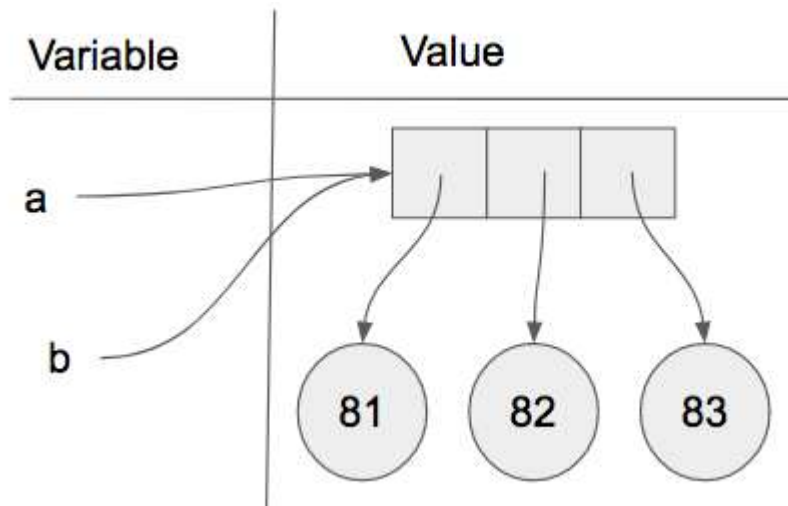
Output

```
{'x': 20}
```

Diagram: Aliasing



✓ Both d1 and d2 point to the **same memory location**.



5. Copying

Copying is the process of creating a **new dictionary object** from an existing dictionary. Python supports **shallow copy** and **deep copy**.

Syntax: `dict.copy()`

Examples

Input: `original = {1:'geeks', 2:'for'}`
`new = original.copy() // Operation`

Output: original: `{1: 'one', 2: 'two'}`
new: `{1: 'one', 2: 'two'}`

6. Shallow Copy

A **shallow copy** creates a new dictionary object, but **nested objects inside the dictionary are shared** between the original and the copied dictionary.

A shallow copy creates a new object which stores the reference of the original elements.

So, a shallow copy doesn't create a copy of nested objects, instead it just copies the reference of nested objects. This means, a copy process does not recurse or create copies of nested objects itself.

Example

`d1 = {"a": 1, "b": [10, 20]}`

```
d2 = d1.copy()
d2["b"].append(30)
print(d1)
print(d2)
```

Output

```
{'a': 1, 'b': [10, 20, 30]}
{'a': 1, 'b': [10, 20, 30]}
```

Diagram: Shallow Copy

```
d1 —▶ {'a':1, 'b': —▶ [10, 20, 30]}
      ▲
d2 —▶ {'a':1, 'b': —|
```

- ✓ Dictionary objects are different
 - ✗ Nested list is shared
-

7. Deep Copy

Definition

A **deep copy** creates a **completely independent copy** of the dictionary, including all nested objects, so modifications in one dictionary do not affect the other. or

A deep copy creates a new object and recursively adds the copies of nested objects present in the original elements.

Example

```
import copy
d1 = {"a": 1, "b": [10, 20]}
d2 = copy.deepcopy(d1)
d2["b"].append(30)
print(d1)
print(d2)
```

Output

```
{'a': 1, 'b': [10, 20]}
{'a': 1, 'b': [10, 20, 30]}
```

Diagram: Deep Copy

```
d1 —▶ {'a':1, 'b': —▶ [10, 20]}
```

d2 —► {'a':1, 'b': —► [10, 20, 30]}

✓ Both dictionary and nested objects are independent

NumPy

NumPy (Numerical Python) is an open-source Python library used for **numerical and scientific computing**, providing support for **multidimensional arrays**, mathematical functions, linear algebra, and efficient data processing.

NumPy includes support for signal processing and linear algebra operations.

NumPy is an open-source project that can be used freely

- Core object: **ndarray**
- Faster than Python lists
- Supports vectorized operations
- Widely used in data science and machine learning

Example

```
import numpy as np
a = np.array([1, 2, 3])
print(a)
```

Output

[1 2 3]

2. Shape

The **shape** of a NumPy array represents the **number of elements along each dimension**, returned as a tuple.

shape() method, comes the flexibility to obtain the dimensions of any Python object. it returns a tuple value that indicates the dimensions of a Python object.

the tuple returned by the shape() method is the actual number of elements that represent the value of the dimension of the object.

- 1D → (n,)

- 2D → (rows, columns)
- Checked using `.shape`

Example

```
a = np.array([[1, 2, 3], [4, 5, 6]])  
print(a.shape)
```

Output

```
(2, 3)  
2 rows × 3 columns  
[1 2 3]  
[4 5 6]
```

3. Slicing

Slicing is the process of extracting a **portion of an array** using index ranges. Or Slicing in Python is a powerful method for extracting a portion (a "slice") of a sequence data type, such as a **string, list, or tuple**. It uses a concise syntax with square brackets and colons to specify the range of elements to be included in the new sequence.

- Syntax: `array[start:end]`
- Works for both 1D and multidimensional arrays

Example

```
a = np.array([10, 20, 30, 40, 50])  
print(a[1:4])
```

Output

```
[20 30 40]  
2D Slicing Example  
b = np.array([[1,2,3],[4,5,6]])  
print(b[:, 1:])
```

Output

```
[[2 3]  
 [5 6]]
```

4. Masking

Masking is the technique of selecting array elements based on a **Boolean condition**.

Masking in Python is a technique used to **select, filter, or modify elements in a data structure based on a boolean condition**. It is a powerful and efficient method primarily used with the `NumPy` and `Pandas` libraries, which support data handling and scientific computing.

How Masking Works

A "mask" is a boolean array (or series) that has the same shape as the original data. Each element in the mask is either `True` or `False`, indicating whether the corresponding element in the original data meets the specified condition.

- `True` means the data element at that position is selected (or "unmasked" and considered valid).
- `False` means the data element is ignored or hidden (or "masked" and considered invalid).

Uses of Masking

- **Data Cleaning:** Handling missing, invalid, or outlier data points.
 - **Conditional Operations:** Applying mathematical operations to only specific subsets of data.
 - **Data Anonymization:** Hiding sensitive information by replacing it with non-sensitive values.
 - **Image Processing:** Selecting specific pixel ranges in an image (e.g., highlighting bone tissue in an X-ray).
- Condition returns True or False
 - Used for filtering data efficiently

Example

```
a = np.array([10, 20, 30, 40])  
print(a[a > 20])
```

Output

```
[30 40]
```

Diagram

Values : 10 20 30 40

Mask : F F T T

Result : 30 40

5. Broadcasting

Broadcasting :NumPy broadcasting is a mechanism that allows NumPy to perform arithmetic operations on arrays of different shapes and sizes. It enables the element-wise operation of arrays without making unnecessary copies of data, thereby enhancing performance and memory efficiency.

- Avoids explicit loops
- Improves performance
- Follows specific shape rules

Example

```
a = np.array([1, 2, 3])
```

```
b = 5
```

```
print(a + b)
```

Output

```
[6 7 8]
 [1 2 3]
+ 5 5 5
-----
 [6 7 8]
```

6. dtype

dtype (data type) specifies the **type of elements** stored in a NumPy array.

- Common types: int32, float64, bool
- Can be explicitly defined

Every ndarray has an associated data type (dtype) object. This data type object (dtype) informs us about the layout of the array. This means it gives us information about:

- Type of the data (integer, float, Python object, etc.)
- Size of the data (number of bytes)
- The byte order of the data (little-endian or big-endian)

- If the data type is a sub-array, what is its shape and data type?

The values of a ndarray are stored in a buffer which can be thought of as a contiguous block of memory bytes. So how these bytes will be interpreted is given by the dtype object.

Example

```
a = np.array([1, 2, 3], dtype=float)
print(a)
print(a.dtype)
```

Output

```
[1. 2. 3.]
float64
```

Files:

1. About Files

A **file** is a named location on secondary storage (such as a hard disk) used to **store data permanently**. Python provides built-in functions to **create, read, write, and manipulate files**.

Python provides built-in functions to open, read, write, and close files. The open() function serves as the primary method for handling files in Python.

Importance Files :

- Data persistence (data is not lost after program ends)
- Large data storage
- Data sharing between programs

File Types

1. **Text Files** – store data as characters (.txt, .csv)
2. **Binary Files** – store data in binary format (.jpg, .pdf, .dat)

2. File Handling in Python; File handling in Python is an important skill for developers. It allows reading, writing, and managing files easily. Python's built-in functions make data storage and processing efficient. We can handle both text and binary files with simple commands, making working with files straightforward in Python.

Basic File Operations

1. Open a file
2. Read or write data
3. Close the file

File Open Modes

Mode	Description
------	-------------

r	Read
w	Write (overwrite)
a	Append
r+	Read and write
b	Binary mode

3. Writing Our First File

Writing to a file means storing data permanently. If the file does not exist, Python **creates it**. If it exists, the content is **overwritten** (in w mode).

To write data to a file, we can use the write() or writelines() methods. The write() method adds a single string, while writelines() can write multiple lines at once.

Example

```
file = open("sample.txt", "w")
file.write("Hello, this is my first file.\n")
file.write("Python file handling.")
file.close()
```

Output (Content of sample.txt)

```
Hello, this is my first file.
Python file handling.
```

4. Reading a File Line-at-a-Time

Reading line-by-line is useful when working with **large files**, as it uses less memory. We can read files in Python using the read(), readline(), or readlines() methods. These methods allow us to read the entire file, a single line, or all lines into a list, respectively.

Example

```
file = open("sample.txt", "r")
for line in file:
    print(line.strip())
file.close()
```

Output

Hello, this is my first file.
Python file handling.

5. Turning a File into a List of Lines

The `readlines()` method reads all lines and stores them as a **list**, where each line is an element.

Reading files and storing their contents in an array is a common task in Python. It is mostly used when we need to process file data line by line or manipulate file data as a list of strings.

The `readlines()` method reads all lines in a file and returns them as a list of strings making it the most efficient way to store file contents in an array.

Example

```
file = open("sample.txt", "r")
lines = file.readlines()
file.close()
print(lines)
```

Output

```
['Hello, this is my first file.\n', 'Python file handling.']
```

6. Reading the Whole File at Once

The `read()` method reads the **entire content** of the file as a single string. Suitable for **small files only**.

To reads the entire content of the file into a list (where each element is a single line, including the newline character `\n`) and then uses a `for` loop to process that list. This is best for small to medium-sized files, as the entire content is stored in memory at once.

Example

```
file = open("sample.txt", "r")
content = file.read()
file.close()
print(content)
```

Output

Hello, this is my first file.
Python file handling.

7. Working with Binary Files

Binary files store data in binary format and are used for **images, audio, video, and executable files**.

Binary files are opened using rb (read binary) or wb (write binary).

Binary File Handling is a process in which we create a file and store data in its original format. It means that if we store an integer value in a binary file, the value will be treated as an integer rather than text. Binary files are mainly used for storing records, just as we store records in a database. It makes it easier to access or modify the records easily.

Example (Copying an Image)

```
source = open("image.jpg", "rb")
target = open("copy.jpg", "wb")
```

```
target.write(source.read())
```

```
source.close()
target.close()
```

The image is copied successfully without data loss.

8. Directories

A **directory (folder)** is a container used to organize files on a storage device.

In Python, directories, commonly known as folders in operating systems, are locations on the filesystem used to store files and other directories. They serve as a way to group and manage files hierarchically.

Python provides several modules, primarily `os` and `os.path`, along with `shutil`, that allows you to perform various operations on directories.

These operations include creating new directories, navigating through existing directories, listing directory contents, changing the current working directory, and removing directories.

Common Directory Operations (Using `os` Module)

```
import os
```

```
os.getcwd()    # Get current directory
os.mkdir("data") # Create directory
os.listdir()    # List files
```

Example

```
import os
print(os.listdir())
```

Output

```
['sample.txt', 'data']
```

9. Fetching Something from the Web

Python can read data directly from the **Internet** using libraries like `urllib`.

The `requests` module allows you to send HTTP requests using Python.

The HTTP request returns a Response Object with all the response data (content, encoding, status, etc)

- **`import requests`** : This line imports the `requests` library, making its functions available in your script.
- **`requests.get(url)`** : This function sends an HTTP GET request to the specified `url`. The GET method is used to request data from a specified resource.
- **`x.text`** : This attribute of the response object `x` returns the content of the response from the server in Unicode format (human-readable text).

Example (Reading a Web Page)

```
import urllib.request
```

```
web = urllib.request.urlopen("http://example.com")  
data = web.read()  
print(data[:100])
```

Output (Partial)

```
b'<!doctype html><html><head>...'
```

Data is read in **binary format**.

Module-4

Modules: Random numbers: the module random from the Python standard library. It contains tools for generating random numbers and other randomized functionality. It is often used in scenarios where unpredictability or randomness is required, such as in simulations, games, cryptography, and data shuffling. The module uses a pseudo-random number generator algorithm to produce seemingly random numbers.

Random module provides functions for generating pseudo-random numbers for integers, floats, and sequences, using the Mersenne Twister algorithm for various distributions like uniform and Gaussian, essential for simulations, games, or testing by seeding for reproducibility. Key functions include random() (float 0-1), randint() (integer in range), choice() (element from sequence), shuffle() (in-place list shuffle), and sample() (selection without replacement).

The random module has a set of methods:

Method Description

seed()	Initialize the random number generator
getstate()	Returns the current internal state of the random number generator
setstate()	Restores the internal state of the random number generator
getrandbits()	Returns a number representing the random bits
randrange()	Returns a random number between the given range

the time module: time module allows to work with time in Python. It allows functionality like getting the current time, pausing the Program from executing. The time module comes with Python's standard utility module,

The time module in Python provides functions for handling time-related tasks.

The time-related tasks includes,

- reading the current time
- formatting time
- sleeping for a specified number of seconds

Example:

```
import time
```

```
print(time.gmtime(0))
```

Output:

```
time.struct_time(tm_year=1970, tm_mon=1, tm_mday=1, tm_hour=0, tm_min=0,
tm_sec=0, tm_wday=3, tm_yday=1, tm_isdst=0)
```

Python time.time() Function

In Python, the time() function returns the number of seconds passed since epoch (the point where time begins).

For the Unix system, January 1, 1970, 00:00:00 at UTC is epoch.

Let's see an example,

```
# import the time module
```

```
import time
```

```
# get the current time in seconds since the epoch
```

```
seconds = time.time()
```

```
print("Seconds since epoch =", seconds)
```

```
# Output: Seconds since epoch = 1672214933.6804628
```

```
#output: Seconds since epoch = 1766589808.5143757
```

Functions :

- import time: You must import the module first to use its functions.
- **Epoch**: A reference point, usually January 1, 1970 (UTC), from which time is measured in seconds.
- time.time(): Returns the current time as seconds since the epoch (a float).
- time.sleep(seconds): Pauses the program's execution for a specified number of seconds.
- time.ctime(): Converts seconds since the epoch into a readable string.
- time.strftime(): Formats time into custom string formats (e.g., %Y-%m-%d %H:%M:%S).
- time.localtime(): Converts seconds since epoch into a struct_time object (a tuple with year, month, day, etc.).

math module :

The **math module** in Python is a built-in library that contains a collection of mathematical functions and constants. The Python math module provides a set of mathematical functions and constants that simplify calculations in programming. It includes functions for arithmetic, trigonometry, logarithms, and more. This module is useful for scientific computing, data analysis, and engineering applications. It also plays a foundational role in numeric programming tasks involving various data types, especially when handling floating-point values.

Importance:

1. Provides built-in functions for complex math operations like square root, power and trigonometry.
2. Offers constants like pi and e, useful in scientific and engineering calculations.
3. Improves accuracy and performance over manual calculations or custom functions.
4. Helps in performing logarithmic and exponential operations easily.
5. Supports real-world applications like physics, statistics, geometry and finance.

Example:

```
import math  
  
print(math.pi)  
  
print(math.e)  
  
print(math.sqrt(2.0))  
  
print(math.sin(math.radians(90))) # sin of 90 degrees
```

Creating your own modules: Creating your own module is a common practice in programming to **organize code into reusable files**. In Python, this is a straightforward process: any file containing valid Python code saved with a `.py` extension is automatically a module.

Steps to Create a Python Module

1. **Create the Module File:** Use a text editor to write your code (functions, variables, or classes).
2. **Save the File:** Save the file with a descriptive name and the `.py` extension. The file name will be the module name.

Writing your own Python modules is easy. Any file containing valid Python code can be imported as a module. Let's assume we have a file named words.py with the following contents:

```
def first_word(my_string: str):  
    parts = my_string.split(" ")  
    return parts[0]  
  
def last_word(my_string: str):  
    parts = my_string.split(" ")  
    return parts[-1]  
  
def number_of_words(my_string: str):  
    parts = my_string.split(" ")  
    return len(parts)
```

he functions defined in the file can be accessed by importing the file:

```
import words  
  
my_string = "Sheila sells seashells by the seashore"  
  
print(words.first_word(my_string))  
print(words.last_word(my_string))  
print(words.number_of_words(my_string))
```

output:
Sheila seashore 6

Namespaces: a namespace is a collection of names.

In Python, we can imagine a namespace as a mapping of every name we have defined to corresponding objects.

It is used to store the values of variables and other objects in the program, and to associate them with a specific name.

This allows us to use the same name for different variables or objects in different parts of your code, without causing any conflicts or confusion.

Types of Python namespace

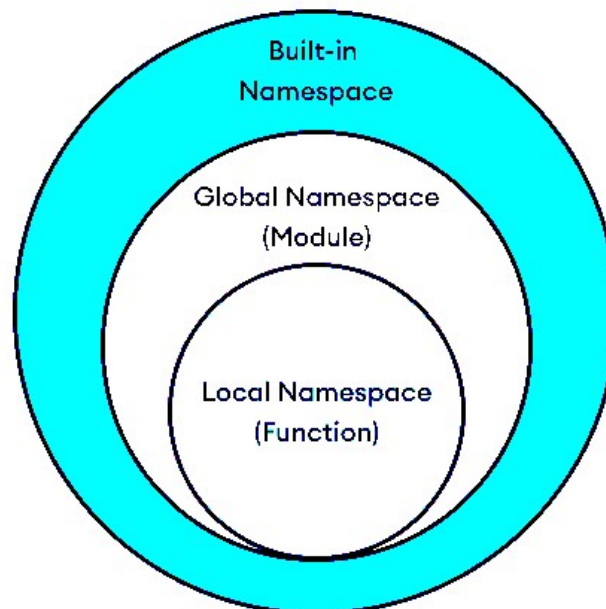
built-in namespace: A namespace containing all the **built-in names** is created when we start the Python interpreter and exists as long as the interpreter runs.

This is the reason that built-in functions like id(), print() etc. are always available to us from any part of the program.

Each module creates its own **global namespace**.

These different namespaces are isolated. Hence, the same name that may exist in different modules does not collide. Modules can have various functions and classes.

local namespace: A **local namespace** is created when a function is called, which has all the names defined in it.



Scope and lookup rules: **scope** defines the region of a program where a variable is accessible, determines the accessibility and visibility of variables within a program. Understanding scope is crucial for writing efficient and bug-free code.

Scoping rules define how the binding of names to objects is governed within a programming language, determining the visibility and lifetime of variables in different parts of a program.

1. The scope that declares a variable directly influences its lifetime and visibility; for example, a file static variable in the C programming language has a lifetime that spans the entire execution of the program but is only visible inside the file where it is declared.
2. Programming languages establish rules for lifetime, visibility, and scope for each name, and compilers typically create data areas associated with various program scopes to simplify memory management. Namespaces provide encapsulation and help manage naming conflicts by creating private areas within a program, allowing procedure and variable names to remain distinct.
3. Binding time refers to the moment when a name is associated with a particular object, and the concept of lifetime applies both to objects and to name-to-object bindings, which may differ in duration.

Lookup rule: In Python, the primary rule for looking up names (variables, functions, etc.) is the **LEGB rule**, which defines the order of scopes Python searches to find an object associated with a given name. the **lookup rule** determines the order in which Python searches for names. The core lookup rule is the **LEGB rule**, which stands for Local, Enclosing, Global, and Built-in scopes.

The lookup order is as follows:

- **L - Local:** Python first checks the current, innermost scope. This includes names defined within the current function or `lambda` expression. A new local scope is created every time a function is called.
- **E - Enclosing:** If the name is not found in the local scope, Python checks the enclosing scope(s). This is applicable when you have nested functions (a function inside another function); the inner function can access names defined in the outer function's scope.
- **G - Global:** If the name is still not found, Python looks in the global scope (module level). This includes names defined at the top level of a script or module, outside of any functions or classes.
- **B - Built-in:** Finally, Python checks the built-in scope. This scope contains names of pre-defined, built-in functions and exceptions, such as `len()`, `print()`, and `NameError`.

Attribute: Attributes in Python are variables that belong to an object and contain information about its properties and characteristics. They can be used to represent details or facts related to the object.

It is a core building blocks of object-oriented programming. They enable objects to have their own data and behavior, allowing them to model real-world entities and scenarios.

By using, attribute, each object can have its unique characteristics and abilities.

all modern programming languages support object-oriented programming and using attributes

Example:

```
class Cat:
```

```
    # Attributes
```

```
    name = 'Mittens'
```

```
    color = 'black'
```

There are two primary types of attributes in Python:

- **Instance Attributes:** These are unique to each instance (object) of a class. They are typically defined within the class's initializer method, `__init__()`, using the `self` keyword. Changing an instance attribute for one object does not affect other objects.
- **Class Attributes:** These attributes belong to the class itself and are shared by all instances of that class. They are defined within the class body, but outside any method, and are useful for storing data that is the same for every instance, such as a constant or a default value.

the dot Operator: The dot operator (.) in Python is a syntax used to access the attributes and methods of an object. It acts as a way to drill down into an object's namespace to retrieve or modify its properties or execute its functions (methods).

syntax : `object.attribute` or `object.method()`.

- **object :** This is the instance of a class, module, or built-in data type (like a string or list) on which you are performing an operation.
- **.(dot operator):** This tells Python to look inside the object's defined scope or namespace for the name that follows.
- **attribute or method :** This is the specific variable (attribute) or function (method) you want to access or call.

Example:

```
class Car:
    def __init__(self, color, model):
        self.color = color
        self.model = model
    def start(self):
        print("Car is started!")
# Usage
car = Car("red", "Toyota")
print(car.color) # Output: red
print(car.model) # Output: Toyota
print(car.start()) # Output: Car is started!
```

Three import statement variants python:

Python offers several variations of the import statement for bringing code from other modules into your current script. The three primary variants are:

Standard Import: This imports the entire module, requiring you to use dot notation to access its contents.

```
import module_name
```

Example:

```
import math
print(math.sqrt(16))
```

This is generally recommended for clarity, as it avoids namespace collisions by making it clear where functions or variables originate.

Import Specific Items: This allows you to import only specific functions, classes, or variables from a module, making them directly accessible without dot notation.

```
from module_name import item1, item2
```

Example:

```
from math import sqrt, pi
```

```
print(sqrt(25))
```

```
print(pi)
```

You can import multiple items by separating them with commas.

Import with an Alias: This allows you to give a module or specific item a different name (alias) within your script, which is especially useful for long module names or for following common community conventions (like import numpy as np).

```
import module_name as alias
```

Example:

```
import numpy as np
```

```
array = np.array([1, 2, 3])
```

You can also alias specific items:

```
from os.path import join as join_path
```

Mutable:

In Python, the concept of mutability refers to whether or not an object's state (its data or content) can be changed after it is created without changing its location in memory.

Objects in Python are classified as either mutable or immutable.

Mutable Objects:

Mutable objects are those whose state or content can be modified after creation on the same memory location. Examples of mutable objects include:

- **Lists:** You can add, remove, or change elements.
- **Dictionaries:** You can add, remove, or change key-value pairs.
- **Sets:** You can add or remove elements.
- **Example—**

```
• L = [1,2,3]
  print(id(L)) #139652074757168

  L.append(4)
  print(L)
  print(id(L)) #139652074757168
```

Immutable Objects:

Immutable objects are those whose state or content cannot be changed after creation.

Examples of immutable objects include:

- **Strings:** You cannot change the characters in a string.
- **Tuples:** You cannot change the elements in a tuple.
- **Numbers:** Integers, floats, etc., cannot be changed.

Example code —

```
# String (immutable)
my_string = "hello"
# my_string[0] = 'H' # This will raise an error because strings are immutable

# Tuple (immutable)
my_tuple = (1, 2, 3)
# my_tuple[0] = 4 # This will raise an error because tuples are immutable
```

Aliasing:

Aliasing happens when multiple variables refer to the same mutable object. Changes made to the object through one alias are reflected in all other aliases, which can lead to unexpected behavior and bugs.

Example code 1:

```
# Create a list and assign it to a variable
original_list = [1, 2, 3]

# Create an alias by assigning the list to another variable
alias_list = original_list

# Modify through alias
alias_list.append(4)

print(original_list)  # [1, 2, 3, 4]
print(alias_list)     # [1, 2, 3, 4]
```

Example code 2:

```
def func(data):
    data.append(4)
a = [1,2,3]
func(a)
print(a)  #[1,2,3,4]
```

Object oriented programming: Classes and Objects :

Object Oriented Programming is a fundamental concept in Python, empowering developers to build modular, maintainable and scalable applications.

OOP is a way of organizing code that uses objects and classes to represent real-world entities and their behavior. In OOP, object has attributes thing that has specific data and can perform certain actions using methods.

Key Features of OOP in Python:

- Organizes code into classes and objects
- Supports encapsulation to group data and methods together
- Enables inheritance for reusability and hierarchy
- Allows polymorphism for flexible method implementation
- Improves modularity, scalability, and maintainability

Class

A class is a collection of objects. Classes are blueprints for creating objects. A class defines a set of attributes and methods that the created objects (instances) can have.

Some points on Python class:

- Classes are created by keyword `class`.
- Attributes are the variables that belong to a class.
- Attributes are always public and can be accessed using the dot (.) operator. Example: `Myclass.Myattribute`

class Dog:

```
species = "Canine" # Class attribute
```

```
def __init__(self, name, age):
```

```
    self.name = name # Instance attribute
```

```
    self.age = age # Instance attribute
```

Objects

An Object is an instance of a Class. It represents a specific implementation of the class and holds its own data.

An object consists of:

- **State:** It is represented by the attributes and reflects the properties of an object.
- **Behavior:** It is represented by the methods of an object and reflects the response of an object to other objects.
- **Identity:** It gives a unique name to an object and enables one object to interact with other objects.

Example:

```
class Dog:
```

```
    species = "Canine" # Class attribute
```

```
    def __init__(self, name, age):
```



```
self.name = name # Instance attribute
```

```
self.age = age # Instance attribute
```

```
# Creating an object of the Dog class
```

```
dog1 = Dog("Buddy", 3)
```

```
print(dog1.name)
```

```
print(dog1.species)
```

Output

Buddy

Canine

Attributes

Attributes are variables defined within a class that represent the characteristics or state of an object. They differentiate one object from another. Attributes are a key part of encapsulation, a core OOP principle that bundles data and the methods that operate on that data into a single, self-contained unit

There are two main types of attributes:

- **Instance Attributes:** These attributes have values that are specific to a particular object instance. They are typically defined within the class's constructor method (e.g., `__init__` in Python or a `constructor` in JavaScript/Java). For example, the `name` and `age` of a specific dog object would be instance attributes.
- **Class Attributes:** These attributes have the same value for all instances of a class. They are defined directly within the class body but outside of any method. For example, a "Dog" class might have a class attribute `species` set to "Canis familiaris", which is true for all dog objects.

Adding methods to our class:

The normal way to add functionality (methods) to a class in Python is to define functions in the class body. There are many other ways to accomplish this that can be useful in different situations.

This is the traditional way

```
class A(object):  
  
    def print_classname(self):  
  
        print self.__class__.__name__
```

The method can also be defined outside the scope of the class. This allows the function “print_classname” to be used as a standalone function and as a method of the class.

```
def print_classname(a):  
  
    print a.__class__.__name__  
  
class A(object):  
  
    print_classname = print_classname
```

Instances as arguments and parameters: A group of connected statements called a **Python function** is used to carry out a logical, computational, or evaluative operation. Functions allow us to divide our program into more manageable and modular sections.

The function declaration is started with the keyword def.

A function name that allows the function to be uniquely identified. **Arguments** that serve as parameters and are used to provide values to functions. They are optional.

Parameters are variables defined in a function declaration. This act as placeholders for the values (arguments) that will be passed to the function.

Arguments are the actual values that you pass to the function when you call it. These values replace the parameters defined in the function.

Although these terms are often used interchangeably, they have distinct roles within a function. This article focuses to clarify them and help us to use Parameters and Arguments effectively.

Parameters

A parameter is the variable defined within the parentheses when we declare a function.

Example:

```
# Here a,b are the parameters  
def sum(a,b):  
    print(a+b)  
  
sum(1,2)
```

Output: 3

Arguments

An argument is a value that is passed to a function when it is called. It might be a variable, value or object passed to a function or method as input.

Example:

```
def sum(a,b):  
    print(a+b)
```

```
# Here the values 1,2 are arguments  
sum(1,2)
```

Output: 3

Converting an instance to a string: Python provides built-in type conversion functions to easily transform one data type into another. This article explores the process of converting objects into strings which is a basic aspect of Python programming.

Since every element in Python is an object, we can use the built-in `str()` and `repr()` methods to convert any built-in object into a string representation.

`str()` method

`str()` is a built-in method so don't need to import any library, we can simply use it over the targeted object.

```
i = 6 # int object
```

```
a = [1, 2, 3, "Hello"] # list object
```

```
# Converting to string
```

```
s1 = str(i)
```

```
print(s1)
```

```
print(type(s1))
```

```
s2 = str(a)
```

```
print(s2)
```

```
print(type(s2))
```

Output: 6

```
<class 'str'>
```

```
[1, 2, 3, 'Hello']
```

```
<class 'str'>
```

repr() method

repr() method is a bit different from str(), it gives a more detailed string representation of an object. Unlike str(), which focuses on making output user-friendly, repr() provides a more technical view which is helpful for debugging and often includes object type or structure and its also customizable. Let's look at some examples,

Example 1: Normal list conversion

```
# Using repr() with a list
```

```
a = [1, 2, 3, "Hello"]
```

```
print(repr(a))
```

```
print(type(repr(a)))
```

Output: [1, 2, 3, 'Hello']

Instances as return values: Functions and methods can return objects. This is actually nothing new since everything in Python is an object and we have been returning values for quite some time. The difference here is that we want to have the method create an object using the constructor and then return it as the value of the method.

The Python return statement marks the end of a function and specifies the value or values to pass back from the function. Return statements can return data of any type, including integers, floats, strings, lists, dictionaries, and even other functions. If no return statement is used, or if a return statement is used without a value, the function will implicitly return None

The return statement in Python has two primary purposes:

1. It immediately **exits a function**.
2. It **sends a value back** to where the function was called.

Any code inside a function after a return statement is executed will not run. If a function doesn't have an explicit return statement, it automatically returns the special value None.

Syntax: `return [value]`

Example:

A function that calculates and returns a value

```
def add(a, b):
```

```
    result = a + b
```

```
    return result
```

The value returned from the function is stored in a variable

```
sum_value = add(5, 3)
```

```
print(sum_value)
```

Outputs: 8

- **What is a module in Python?**

- A module is a Python file (`.py` extension) containing functions, classes, and variables that can be imported and reused in other programs, helping to organize code and prevent name clashes.

- **How do you create and use your own module?**

- Create a file (e.g., `calc.py`) with your code. In another Python script, use `import calc` to access its functions and variables (e.g., `calc.add(a, b)`).

- **Explain the three variants of the import statement (`import`, `from import`, `from import *`) and when to use each.**

- `import module_name`: Imports the entire module, requiring you to use the module name as a prefix (e.g., `math.sqrt(25)`).
- `from module_name import function_name` (or multiple functions/classes): Imports only specific components into the current namespace, so you can use them directly without the prefix (e.g., `sqrt(25)`).

- `from module_name import *`: Imports all components from a module into the current namespace. This is generally discouraged as it can cause name collisions (overwriting existing variable/function names).
- **What are namespaces, and how do they relate to scope and lookup rules (LEGB rule)?**
- A namespace is a mapping from names to objects. Scopes define where a name is recognized without a prefix. Python uses the LEGB rule (Local, Enclosing, Global, Built-in) to look up a name: it checks the local scope first, then any enclosing functions, then the global scope (current module), and finally the built-in names.
- **Explain the use of the `global` and `nonlocal` keywords.**
- `global` allows modifying a variable in the global (module-level) scope from within a function.
- `nonlocal` allows modifying a variable in an enclosing (but non-global) scope.
- **How do you access attributes of an object or module?**
- You use the dot operator (`.`), such as `module_name.attribute_name` or `object_name.method_name`.

Important Questions:

- What is the difference between mutable and immutable data types? Provide examples.
- Explain "aliasing" and how it behaves differently with mutable vs. immutable objects.
- How can you avoid aliasing issues when you need a separate, modifiable copy of a mutable object?
- What are classes and objects in Python OOP?
- What is the `__init__` method and what is its purpose?
- What is the `self` parameter used for?
- How do you convert an instance to a string for printing?
- How are instances passed as arguments to functions or methods, and returned as values?

Module-5

Object oriented programming: Objects are mutable

Object Oriented Programming is a programming language model in which programs are organized around data, objects rather than functions and logic. This in other words means that the software design process is by applying data structures. However, the data structure in the OOP becomes an object that includes data and functions creating relationships between objects thus the objects can inherit characteristics from other objects.

Python is known for being an OOP language meaning that its framework of programming is focused on objects, allowing the user to create and manipulate attributes and methods so the program can be consistent and efficient by having the required structures. One of Python's core is based from the PyObject data structure and the data types are inherited from PyObjects.

Mutable objects are objects whose values can change. Examples of mutable types in Python are list, dictionary, set and user-defined classes.

The use of mutable objects is recommended when there is a need to change the size or content of the object.

an object is instantiated, it is assigned a unique object id. The type of the object is defined at the runtime and it can't be changed afterward. However, its state can be changed if it is a mutable object.

built-in mutable types include:

- **Lists:** `list.append()`, element assignment (e.g., `my_list[0] = 10`).
- **Dictionaries:** `dict.update()`, adding or changing key-value pairs.
- **Sets:** `set.add()`, `set.remove()`.
- **Byte arrays.**
- **User-defined classes** (by default).

dictionary that are mutable i.e., we can make changes in the Dictionary.

```
my_dict = {"name": "Ram", "age": 25}
```

```
new_dict = my_dict
```

```
new_dict["age"] = 37
```

```
print(my_dict)
```



```
print(new_dict)
```

Output

```
{'name': 'Ram', 'age': 37}
```

```
{'name': 'Ram', 'age': 37}
```

Sameness: In Python, "sameness" refers to two distinct concepts: object identity (are they the exact same object in memory?) and value equality (do they have the same contents?). The operators `is` and `==` are used to check for these.

1. Object Identity (`is` operator)

Object identity, also known as shallow equality, determines if two variables refer to the exact same object in memory. The built-in `id()` function returns a unique identifier for an object, which often corresponds to its memory address in the standard C Python interpreter.

Operator: `is`

Purpose: Checks if two variables point to the same memory location.

Example:

```
a = [1, 2, 3]
```

```
b = a      # b is an alias of a
```

```
c = [1, 2, 3] # c is a new list object with same content
```

```
print(a is b) # Output: True (same object)
```

```
print(a is c) # Output: False (different objects)
```

```
print(id(a) == id(b)) # Output: True (their IDs are the same)
```

2. Value Equality (`==` operator)

Value equality, or deep equality, checks if two objects have the same value or contents, regardless of where they are stored in memory. For built-in types (like numbers, strings, lists, and dictionaries), Python automatically compares their contents.

Operator: `==`

Purpose: Checks if the values of two variables are equal.

Example:

```
a = [1, 2, 3]
b = a
c = [1, 2, 3]
print(a == b) # Output: True (same value)
print(a == c) # Output: True (same value)
# Different types can sometimes be equal by value (e.g., int and float)
print(1 == 1.0) # Output: True
print("5" == 5) # Output: False (types matter for strings/numbers)
```

Copying: In Python, copying mutable objects like lists or dictionaries. There are two types of copies: shallow copy and deep copy.

Shallow Copy

A shallow copy creates a new top-level container object but populates it with references to the inner objects from the original.

Behavior: Changes to immutable elements (like integers, strings) in the copy will not affect the original. However, changes to mutable nested elements (like nested lists or dictionaries) will affect the original, as they share the same reference.

Methods:

```
Slicing: new_list = original_list[:]
copy() method: new_list = original_list.copy()
list() constructor: new_list = list(original_list)
copy.copy() function: import copy; new_list = copy.copy(original_list)
import copy
original_list = [1, [2, 3], 4]
shallow_copy = original_list.copy()
# Modify a top-level element (immutable)
shallow_copy[0] = 99
print(original_list[0]) # Output: 1 (original is unchanged)
# Modify a nested element (mutable)
shallow_copy[1][0] = 88
```

```
print(original_list[1][0]) # Output: 88 (original IS changed because the nested list is shared)
```

Deep Copy:

A deep copy creates a fully independent clone of the original object and all its nested objects recursively. No references are shared between the original and the copy.

Behavior: Changes made to the copy, at any level, do not affect the original.

Method:

`copy.deepcopy()` function: You must import the copy module.

```
import copy
```

```
original_list = [1, [2, 3], 4]
```

```
deep_copy = copy.deepcopy(original_list)
```

```
# Modify a nested element in the deep copy
```

```
deep_copy[1][0] = 88
```

```
print(original_list[1][0]) # Output: 2 (original is unchanged)
```

```
print(deep_copy[1][0])    # Output: 88 (copy is changed)
```

Inheritance: Inheritance is a fundamental concept in object-oriented programming (OOP) that allows a class (called a child or derived class) to inherit attributes and methods from another class (called a parent or base class).

It is one of the core principles of object-oriented programming (OOP), allowing classes to derive properties and behavior from various other classes. In Python, inheritance promotes code reusability, modularity, and logical structure, making it easier to manage complex software systems.

Inheritance in Python enables a new class (child or derived class) to inherit attributes and methods from an existing class (parent or base class). This means you can define common functionality in a parent class and extend or customise it in child classes without rewriting code.

Example:

```
class Animal:
```

```
    def speak(self):
```

```
return "Animal speaks"
```

```
class Dog(Animal):
```

```
def bark(self):
```

```
    return "Dog barks"
```

Here, the Dog class inherits the speak() method from Animal, while also introducing its method bark().

Pure function: A pure function in programming is a function that produces the same output for the same input and has no side effects (it does not modify any external state). [1.1, 1.2, 1.3]

The function creates a new Time object, initializes its attributes, and returns a reference to the new object. This is called purefunction Because it does not modify any of the objects passed to its arguments and it as argument and it has no effect, like displaying a value or getting user input, other than returning a value.

Here is simple prototype of add_time:

```
def add_time(t1,t2):
```

```
    sum=Time()
```

```
    sum.hour=t1.hour+t2.hour
```

```
    sum.minute=t1.minute+t2.minute
```

```
    sum.second=t1.second+t2.second
```

```
    return sum
```

Modifiers: In Python, access modifiers are used to control how class variables and methods can be accessed. There are three types: public, protected, and private.

1. Public Members

By default, all class members in Python are public. This means you can access them from anywhere — both inside and outside the class.

```
class Course:
    def __init__(self):
        self.title = "Python Programming"

obj = Course()
print(obj.title) # Output: Python Programming
```

Here, title is a public variable. You didn't need to do anything special to access it — it's open to all.

2. Protected Members

When you add a **single underscore** before a variable or method (like `_name`), you're saying, "Hey, this is for internal use or subclasses only."

```
class Course:
    def __init__(self):
        self._instructor = "Harish"

class ExtendedCourse(Course):
    def show_instructor(self):
        print(self._instructor)

obj = ExtendedCourse()
obj.show_instructor() # Output: Harish
```

Python won't stop you from accessing it directly, but by using `_`, you're telling other developers to treat it with care.

3. Private Members

If you want to really hide a variable, prefix it with **double underscores** (like `__version`). This triggers **name mangling**, which makes it harder (but not impossible) to access from outside the class.

```
class Course:
    def __init__(self):
```

```
self.__version = "3.11"
```

```
def show_version(self):  
    print(self.__version)
```

```
obj = Course()
```

```
obj.show_version() # Output: 3.11
```

```
print(obj.__version) # This will raise an error  
print(self.__version)
```

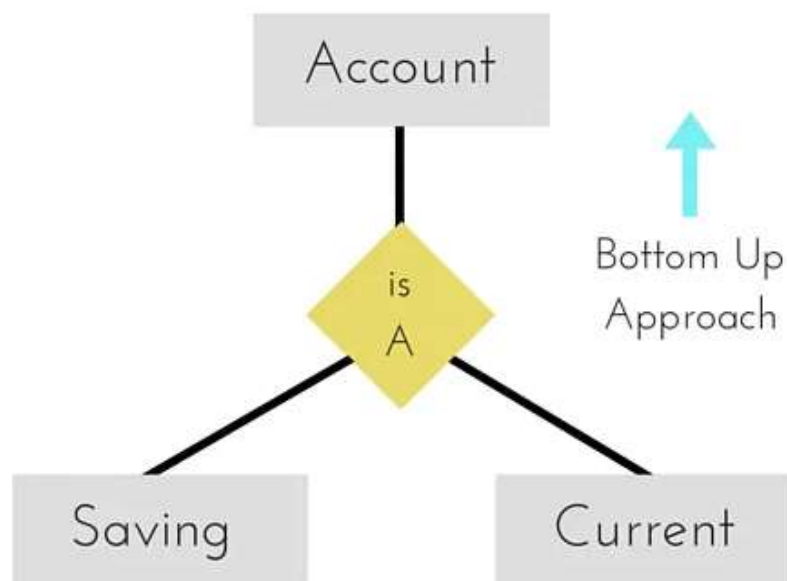
```
def show_version(self):
```

Trying to access `__version` directly will fail. But if you really need to, you can do this (not recommended though):

```
print(obj._Course__version) # Output: 3.11
```

Generalization is a bottom-up approach in which two lower level entities combine to form a higher level entity. In generalization, the higher level entity can also combine with other lower level entities to make further higher level entity.

It's more like Superclass and Subclass system, but the only difference is the approach, which is bottom-up. Hence, entities are combined to form a more generalised entity, in other words, sub-classes are combined to form a super-class.



For example, **Saving** and **Current** account types entities can be generalised and an entity with name **Account** can be created, which covers both.

Features of Generalization in OOPs

- **Generalization in OOPS** involves creating a superclass that encapsulates shared attributes and methods across related classes.
- It promotes code reusability by allowing subclasses to inherit common functionality.
- Generalization supports abstraction by focusing on the common characteristics of objects.
- It enables polymorphism, where subclasses can implement specialized behaviors while sharing a common interface.
- Generalization establishes a clear class hierarchy, making the code more efficient, modular, and easier to maintain

operator overloading : Python, which allows the same operator to have different meanings depending on the context is called **operator overloading**

to Overload the Operators in Python?

We overload the operators in Python by defining special methods inside a class. These methods start and end with double underscores, like `__add__`, `__sub__`, `__mul__`, etc.

Create a class with necessary attributes.

Add special methods like `__add__`, `__sub__`, `__eq__`, etc.

Write logic to return a result or new object.

Example:

class Point:

```
def __init__(self, x, y):
    self.x = x
    self.y = y
def __add__(self, other):
    return Point(self.x + other.x, self.y + other.y)
def __str__(self):
    return f'({self.x}, {self.y})'
```

```
# Creating objects
p1 = Point(3, 4)
p2 = Point(1, 2)
# Using overloaded +
result = p1 + p2
print(result) # Output: (4, 6)
```

Advantages of Python Operator Overloading

Operator overloading in Python offers several benefits, which we have mentioned below.

1. Improved Readability

As you overload boolean operators, you can create natural syntax and semantics for your class and make your code and intent of operations easier.

2. Concise Code

Overloading boolean operators makes your code concise and expressive, so you can use operators to perform custom operations on your objects.

3. Expressive Code

You can write code similar to mathematical and real-world operations to make your code more intuitive, making it easier for you to understand and maintain it.

4 Improved Functionality

Overloading boolean operators enables you to add new functionality to a class. For example, performing 'or' operation or logical operation on the class instance.

5. Consistency

Operator overloading helps your custom class work just like built-in types. This keeps your code uniform and makes sure operators behave the same way everywhere.

Polymorphism

Polymorphism is a key idea in Object-Oriented Programming (OOP). The word "polymorphism" comes from Greek. "Poly" means "many." "Morph" means "forms." So, polymorphism means "many forms."

In Python, polymorphism allows one thing to do different actions. This could be a function, a method, or an operator. The action it takes depends on the type of data it works with. This means you can call the same method on different objects. Each object will then respond in its own special way.

- With Built-in Functions: Python's own functions often use polymorphism.
- With Class Methods: Different classes can have methods that share the same name.
- With Inheritance (Method Overriding): Child classes can change methods they get from their parent classes.

Example:

```
class Animal:
```

```
    def sound(self):
```

```
        raise NotImplementedError("Subclass must implement abstract method")
```

```
class Dog(Animal):
```

```
    def sound(self):
```

```
        return "Woof!"
```

```
class Cat(Animal):
```

```
    def sound(self):
```

```
        return "Meow!"
```

```
# A list of Animal objects
```

```
animals = [Dog(), Cat()]
```

```
for animal in animals:
```

```
    # The correct sound() method is called based on the object's actual type at runtime
```

```
    print(animal.sound())
```

Output:

Woof!

Meow!

- **Compile-time polymorphism** is resolved at compile time. Examples include function/method overloading and operator overloading.
- **Run-time polymorphism** (dynamic polymorphism) is resolved at runtime. This is achieved through method overriding and virtual functions (in languages like C++) or interface implementations, where the actual method called depends on the object's type, not the reference type.

Exceptions: An exception is an unexpected event that occurs during program execution. For example,

```
divide_by_zero = 7 / 0
```

The above code causes an exception as it is not possible to divide a number by 0.

Errors that occur at runtime (after passing the syntax test) are called **exceptions** or **logical errors**.

For instance, they occur when we

- try to open a file(for reading) that does not exist (`FileNotFoundError`)
- try to divide a number by zero (`ZeroDivisionError`)
- try to import a module that does not exist (`ImportError`) and so on

Catching Exceptions: In Python, you catch exceptions using the `try...except` block, which allows you to monitor a section of code for errors and execute specific handling code if an exception occurs, preventing the program from crashing.

Exception handling in Python is a process of resolving errors that occur in a program. This involves catching exceptions, understanding what caused them, and then responding accordingly. Exceptions are errors that occur at runtime when the program is being executed. They are usually caused by invalid user input or code that is invalid in Python. Exception handling allows the program to continue to execute even if an error occurs.

The most fundamental way to handle exceptions is with `try` and `except`.

This involves placing code that might raise an exception within the `try` block. If an exception occurs, the code in the corresponding `except` block is executed. For example:

```
try:
    # Code that might raise an exception
    result = 10 / 0
except ZeroDivisionError:
    # Code to handle the specific exception
    print("Error: You cannot divide by zero.")
# Program continues execution after the except block
print("Execution continues.")
```

Output:

Error: You cannot divide by zero.
Execution continues.

the use of try , except (or catch) , else , and finally blocks.

Code that might raise an exception is placed in the try block. The except (or catch) block handles specific exceptions if they occur. The else block (in Python, for example) runs only if no exception was raised in the try block. The finally block always runs, regardless of whether an exception occurred, and is used for essential cleanup tasks like closing files or releasing resources.

The try , except (or catch) , else , and finally blocks are used for handling exceptions in many programming languages.

- **try block:** This block contains the code that might potentially raise an exception during execution [1].
- **except (or catch) block:** This block catches and handles specific exceptions that are raised within the corresponding try block [1].
- **else block:** (Specific to Python, and some other languages) The code within this block runs only if no exception was raised in the try block [1].
- **finally block:** This block always executes, whether an exception occurred or not. It is typically used for necessary cleanup tasks, such as closing open files or releasing system resources

```
try:
    # Attempt to perform an action that might cause an error
    number = int(input("Enter a number: "))
    result = 10 / number
```

```
except ValueError:
    # Handle specific error if the user input is not a valid integer
    print("Invalid input. Please enter an integer.")
except ZeroDivisionError:
    # Handle specific error if the user enters 0
    print("Cannot divide by zero.")
else:
    # Code to run only if no exceptions occurred in the try block
    print(f"The result is {result}")
finally:
    # Code that always runs, regardless of the outcome
    print("Execution finished.")
```

Raising your own exceptions: In Python, you can raise your own exceptions using the `raise` keyword, either with built-in exception types or by defining custom exception classes that inherit from `Exception`.

Raising a Built-in Exception

You can use the `raise` statement to trigger one of Python's standard, built-in exceptions with a custom error message when a specific condition is met.

```
def check_positive(number):
    if number < 0:
        # Raise a ValueError if the condition is not met
        raise ValueError("Sorry, number must be positive")
    return number

# Example usage:
try:
    check_positive(-5)
except ValueError as e:
    print(f"Caught an exception: {e}")
```

Output:

Caught an exception: Sorry, number must be positive

Defining and Raising a Custom Exception

For application-specific errors, you can create a custom exception by defining a new class that inherits from the base `Exception` class or a more specific subclass like `ValueError` or `TypeError`.

1. Define the custom exception class

```
class InvalidAgeError(Exception):
```

```
    """Exception raised for invalid age values."""
```

```
    def __init__(self, age, message="Age is outside the allowed range (0-120)":
```

```
        self.age = age
```

```
        self.message = message
```

```
        # Call the base class constructor with the message
```

```
        super().__init__(self.message)
```

2. Use the custom exception in a function

```
def set_age(age):
```

```
    if not (0 <= age <= 120):
```

```
        # Raise the custom exception
```

```
        raise InvalidAgeError(age)
```

```
    print(f"Age set to {age}")
```

3. Handle the custom exception using try-except

```
try:
```

```
    set_age(150)
```

```
except InvalidAgeError as e:
```

```
    print(f"Error: Invalid age provided: {e.age}. {e.message}")
```

```
except Exception as e:
```

```
    # General handler for any other potential exceptions
```

```
    print(f"An unexpected error occurred: {e}")
```

Output:

Error: Invalid age provided: 150. Age is outside the allowed range (0-120)

Important questions:

- Explain the difference between mutable and immutable objects.
- What is "sameness" or object identity in OOP, and how does it differ from equality?
- Can you explain the concept of "pure functions" and how they relate (or don't relate) to object-oriented programming?
- What are "modifiers" (access specifiers) in OOP, and what is their role in inheritance?
- How do you handle copying objects in an object-oriented language, and what is the difference between a shallow and a deep copy?
- Explain "generalization" (the "is-a" relationship) in the context of inheritance
- What is operator overloading, and how does it relate to polymorphism?
- Differentiate between compile-time (static) and run-time (dynamic) polymorphism.
- What is exception handling, and why is it important?
- Explain the use of try, except (or catch), else, and finally blocks.
- When would you raise your own (custom) exceptions

*****ALL THE BEST*****